

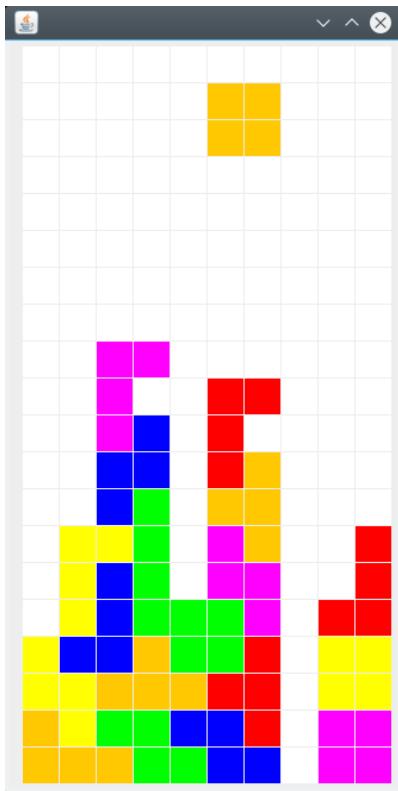
Lab Handout for Relaxed Tetris

This lab is divided in two parts and concerns a variation on the classic game of “Tetris” which is called “Relaxed Tetris”.

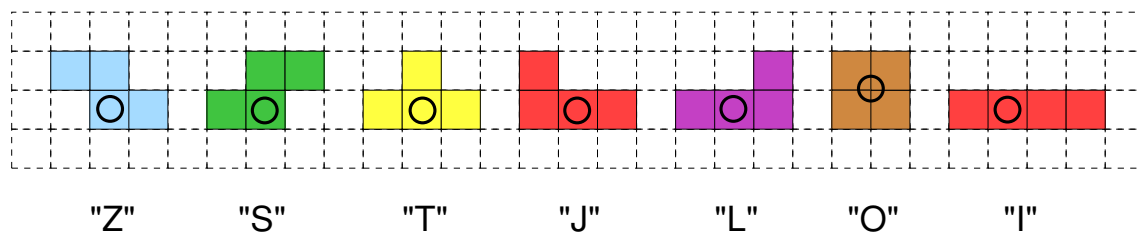
The tetris pieces (called Tetrominos) are controlled using the keyboard (in this case, the arrow keys).

But, instead of requiring quick action and reflexes, this variation is called “relaxed mode”, and the piece only move when the user provides input. The piece go down only when the user presses (5). Pieces move left (4) or right (6), and rotate left/counter-clockwise (7) or right/clockwise (8). Pieces can be landed (moved down as much as possible) using the space bar.

The code includes a graphical user interface so you can play it:



In Tetris, there are seven tetrominoes:



Each tetromino has an axis of rotation illustrated above using a circle and an orientation (i.e. North, East, South, West). Doing this lab you may discover that there is actually a layer of ambiguity in how this concept of axis of rotation can be understood.

Some versions of tetris push pieces to allow rotations in certain cases. This version is simpler, and if an operation (move or rotation) would cause two pieces to overlap, the operation is simply ignored and nothing happens.

In this version of Tetris the moves on a piece are finished when the bottom of the piece is touching any other piece or the bottom of the board.

Part 0: Getting Started

To get started, download the LabWeek8.zip file from the lecture schedule on the course website. There is a main that can be used to run the game. A simple GUI should appear on your screen, and you should be able to play the game.

Remember, however, that at this stage the game contains a number of bugs and missing features.

For example,

pieces move in the opposite direction from what you are expecting, and the space bar does not work.

Simple mechanical set up (similar to all other labs)

- Download the LabWeek8.zip file from the course website
- Make a new Java Eclipse project.
- Copy all of the files into /src in that project.
- Make sure you do not accidentally introduce a module-info.java file
- Try to run the tests. You will get a compile time error (JUnit not present)
- Add JUnit following the Eclipse hint.
- By running the tests at this stage you should see that only `OverlapOp.testOverlapAlone2` is passing. In Eclipse you can run all the test by right clicking on the tests package and selecting “run as JUnit test”. If you do so, you should read `Runs 58/58 Errors 22 Failures 35 Since 22+35=57`, there is a single passing test.

Understanding the Code

Each piece is implemented as a separate class which extends the abstract class `Tetromino`.

You should find the following Java packages

- The `swen221/tetris/gui/` package contains the graphical user interface and the main method. NOTE: you do not need to modify any code in this package. While you may be able to complete this assignment without understanding the inner working of this package, I advise you to read it, and take note on how the code uses nested classes and sub methods to divide responsibilities. Also note how we use a map to represent key bindings.

- The `swen221/tetris/logic/` package contains the class `Game` encoding the logic of a game of Tetris, and the class `Board` representing the current state of the board.

- The `swen221/tetris/tetromino/` package contains the interface `Tetromino` and the concrete implementations for all tetrominos.

- The `swen221/tetris/tests/` packages contains many JUnit tests to check your implementation of the game.

Reading the code

Spend some time familiarizing yourself with this code. It may be faster to simply read all the code before starting to work, so that you can familiarize yourself with the code structure, instead of having to discover it step by step while solving the various tasks.

This code uses a lot of streams optionals and lambdas. Try to learn from this coding style.

Note that the code inside of the tests is designed to be simpler than the application code. For example the code of `TetrominoOk.equalTetromino` would be very different and much more compact if it was part of the application code.

Part 1: Move left and landing

The code of `Tetromino.copyOnBoard` needs to be fixed.

Methods `Tetromino.x` and `Tetromino.y` return the coordinates of each of the 4 squares of every tetronimo, and they should be used to write the tetronimo on the board; but only method `Tetromino.y` is currently used. After fixing the code of `Tetromino.copyOnBoard` you can re-run the tests and you should see

Runs 58/58 Errors 26 Failures 28

Since $26+28=54$, there are now 4 passing tests.

They should be `OverlapOp.testOverlapAlone2`, `STest.testPut`, `TTest.testPut`, `ITest.testPut`.

We now focus on `ITest.java`. Some of those tests display a useful testing technique that you can learn going forward: to force Game to produce only I pieces, we override one of its methods.

If you run the game now, moving a piece left will make it go right.

Some tests are failing inside of `ITest.java` because of this. Fix this simple bug where moving a piece left makes it go right.

By re-running the tests you should see

Runs 58/58 Errors 26 Failures 26.

You will see 2 additional tests pass now: `ITest.moveLeft` and `ITest.moveLeftTwice`.

Another test is failing because landing is not implemented.

Implement `Game.landing`. By rerunning the tests you should see

Runs 58/58 Errors 26 Failures 24.

You will see that 2 additional tests pass now: `ITest.moveLanding` and `ITest.moveLeftLanding`.

You will see that 2 tests are still not passing, since they rely on `Tetromino.rotateLeft`. Complete `Tetromino.rotateLeft` by calling `Tetromino.rotateRight` 3 times.

By re-running the tests you should see

Runs 58/58 Errors 26 Failures 13.

You will see that these tests suites are now all green: `ITest`, `TTest` and `STest`.

Also `TestTetrominoOk.testI`, `TestTetrominoOk.testS` and `TestTetrominoOk.testT` should now be passing.

Part 2: Tetromino.overlap

You should now see that most of the tests in `OverlapOk.java` are failing.

These tests show an interesting technique for writing many tests without cutpasting most of the test logic.

Complete `Tetromino.overlap` to make all of the tests inside `OverlapOk` pass.

HARD: To get the full tutor mark for Part 2, you need to complete `Tetromino.overlap` using elegant stream operations. The method body should start with `“return Board.rangeT()”` and work by using the methods of streams and optionals, without needing to rely on `for-while-if-switch-try, ?:` etcetera. The tutors will check for this during the tutor marking. If you complete this part correctly, but using old style code with `for-while-if` and similar statements, you will still get some marks for it, but not the full amount.

By re-running the tests you should see

Runs 58/58 Errors 26 Failures 6.

You will see that the following tests now pass:

`OverlapOk.testOverlapWithBar1`, `OverlapOk.testOverlapWithBar2`,

`OverlapOk.testNoOverlapAlone0`,

`OverlapOk.testNoOverlapAlone1`, `OverlapOk.testNoOverlapAlone2` and

`OverlapOk.testNoOverlapAlone4`.

Submission

Your lab solution should be submitted electronically via the online submission system, linked from the course homepage. You need to submit a zip with all the needed source files as for the other labs.

Do not include your testing files, only your source files.

The submission system will guide you to submit the right files.

You can submit as many times as you want before the deadline, with no penalty.

The last mark will be saved.

Make sure to press 'run tests'.

If you do not see a clear screen showing your mark, you have not completed the submission.

Note that this is only 50% of the lab marks, the other 50% is obtained by engaging with the Tutors for the in person evaluation. They will ask you questions about this activity and you have to answer with clear good terminology.

Note: Failure to engage with the marking system and the tutors in person marking may cause zero marks to be awarded.

Deadlines

- Automatic Marking: Before Friday of week 8
- Tutor mark: any lab slot of Week8 or Week 9